GMERT algorithm

Paolo Colussi

2025-11-04

Model specification and notation

Let y_{ij} denote the binary response for the i-th observation ($i = 1, \ldots, n_j$) within cluster j ($j = 1, \ldots, n$). Conditional on the random effects b_j, the responses y_{ij} are assumed independent and follow a Bernoulli distribution with parameter μ_{ij}, that is,

$$y_{ij} \mid b_j \sim \text{Bernoulli}(\mu_{ij}), \qquad \mathbb{E}(y_{ij} \mid b_j) = \mu_{ij}, \qquad \text{Var}(y_{ij} \mid b_j) = \mu_{ij}(1 - \mu_{ij}).$$

The relationship between the conditional mean μ_{ij} and the linear predictor η_{ij} is specified through the canonical logit link function,

$$\eta_{ij} = g(\mu_{ij}) = \log\left(\frac{\mu_{ij}}{1 - \mu_{ij}}\right).$$

In the generalized mixed-effects regression tree (GMERT) model, the linear predictor is composed of a fixed component $f(x_{ij})$ and a random component $z_{ij}^\top b_j$, such that

$$\eta_{ij} = f(x_{ij}) + z_{ij}^\top b_j, \qquad b_j \sim \mathcal{N}(0, D),$$

where $f(x_{ij})$ is an unknown function representing the fixed effects, estimated nonparametrically by means of a regression tree, b_j is a vector of cluster-specific random effects with covariance matrix D and z_{ij} is the vector for the random effects.

Algorithm

The GMERT algorithm is the ML-based PQL-EM-algorithm in which we replace the linear structure used to estimate the fixed part of the model by a standard tree structure. The algorithm is as follows:

Step 0

Let $M = 0$. Starting from initial values for the conditional means $\hat{\mu}_{ij}^{(0)}$ (for binary data, one convenient choice is $\hat{\mu}_{ij}^{(0)} = 0.25$ if $y_{ij} = 0$ and $\hat{\mu}_{ij}^{(0)} = 0.75$ if $y_{ij} = 1$), define the linearized (working) responses and the associated weights according to the penalized quasi-likelihood (PQL) scheme. Specifically, set

$$\tilde{y}_{ij}^{(0)} \;=\; g\big(\hat{\mu}_{ij}^{(0)}\big) \;+\; \big(y_{ij} - \hat{\mu}_{ij}^{(0)}\big) \odot g'\big(\hat{\mu}_{ij}^{(0)}\big), \qquad W_j^{(0)} \;=\; \text{diag}\big(w_{ij}^{(0)}\big),$$

with $g(\cdot)$ the logit link, $g'(\mu_{ij}) = [\mu_{ij}(1 - \mu_{ij})]^{-1}$, and $w_{ij}^{(0)} = \big(v_{ij}\, g'(\hat{\mu}_{ij}^{(0)})^2\big)^{-1}$. For Bernoulli outcomes, $v_{ij} = \mu_{ij}(1 - \mu_{ij})$, which yields the familiar working weights $w_{ij}^{(0)} = \hat{\mu}_{ij}^{(0)}\big(1 - \hat{\mu}_{ij}^{(0)}\big)$. Using $(\tilde{y}_{ij}^{(0)}, W_j^{(0)})$, fit a *weighted* linear mixed-effects pseudo-model to obtain initial estimates $\hat{\sigma}^{2(0)}$ and $\hat{D}^{(0)}$. These values initialize the subsequent weighted MERT iterations that update the tree-based fixed part and the random effects under the PQL framework.[1]

[1] See GMERT algorithm initialization and binary special case for the definitions of $\tilde{y}$ and w_{ij}.

Outer loop

After obtaining the initial estimates of the working responses $\tilde{y}_{ij}^{(0)}$, the weights $W_j^{(0)}$, and the variance components $\hat{\sigma}^{2(0)}$ and $\hat{D}^{(0)}$, the outer loop is initialized by setting $M = 0$. This loop governs the successive updates of the working quantities $(\eta_{ij}, \mu_{ij}, \tilde{y}_{ij}, W_j)$ within the PQL framework. At each outer iteration, the algorithm calls the inner weighted MERT procedure to update the fixed and random components, after which the working responses and weights are recomputed. The outer loop proceeds until convergence of the linear predictors η_{ij}.

Start Inner loop

While the outer loop governs the PQL updates of the working responses and weights, the inner loop performs the weighted mixed-effects regression tree estimation using the current pseudo-data. For each outer iteration M, and while the generalized log-likelihood (GLL) has not yet converged, the following steps are repeated.

Step 1

At the beginning of the m-th inner iteration, compute the adjusted working responses

$$\tilde{y}_{ij}^{*(m)} = \tilde{y}_{ij}^{(M)} - Z_j\, \hat{b}_j^{(m-1)}, \qquad j = 1, \ldots, n.$$

A weighted regression tree is then fitted to the data $(\tilde{y}_j^{*(m)}, X_j, W_j^{(M)})$ using a standard tree algorithm, thereby yielding an updated estimate of the fixed component, denoted $\hat{f}^{(m)}(X_j)$. The random effects are subsequently updated by the empirical best linear unbiased predictor (BLUP),

$$\hat{b}_j^{(m)} = \hat{D}^{(m-1)} (W_j^{(M)1/2} Z_j)^\top \hat{V}_j^{-(m-1)} \Big(W_j^{(M)1/2} \tilde{y}_j^{(M)} - W_j^{(M)1/2} \hat{f}^{(m)}(X_j) \Big),$$

In R:

```
# b_fun: updates random effects b_i given current D and provided V_j
#        w is a LIST with one diagonal weight matrix W_j per cluster
b_fun <- function(G,    # number of clusters
                  Z,    # N x q random-effects design (stacked Z_j)
                  Vj,   # list length G with V_j matrices (n_j x n_j)
                  D,    # q x q covariance of random effects
                  idx,  # list length G with row indices per cluster
                  y_t,  # length N pseudo-response (y_tilde)
                  fhat, # length N fixed-part prediction
                  w     # length N weights
){
  b <- matrix(0, G, ncol(Z))
  for (g in seq_len(G)) {
    Zj   <- Z[idx[[g]], , drop = FALSE]
    Wj_sq <- diag(sqrt(w[idx[[g]]]))                                # W_j^{1/2}

    Zj_w <- Wi_sq %*% Zj                                            # W^{1/2} Z_j
    rhs  <-  Wj_sq %*% y_t[idx[[g]]] - Wj_sq %*% fhat[idx[[g]]]    # W^{1/2} (y - fhat)

    b[g, ] <- D %*% t(Zj_w) %*% solve(Vj[[g]]) %*% rhs              # b_i = D (Z_w)^T V_j^{-1} rhs
  }
  b
}
```

where

$$\hat{V}_j^{(m-1)} = W_j^{(M)1/2} Z_j\, \hat{D}^{(m-1)} (W_j^{(M)1/2} Z_j)^\top + \hat{\sigma}^{2(m-1)} I_{n_j}.$$

In R:

```r
# Vi_fun: builds list of V_j = Z_j D Z_j^T + sigma^2 I_{n_j}
Vj_fun <- function(Z,    # Z   : N x q random-effects design
                   D,    # D   : q x q covariance of random effects
                   s2,   # s2  : scalar sigma^2 (residual variance; homoskedastic)
                   G,    # G   : number of clusters
                   idx,  # idx : list of row indices per cluster
                   w     # length N weights
                   ){
  Vi <- list()
  for (g in seq_len(G)) {
    Zj <- Z[idx[[g]], , drop = FALSE]                                    # Z_j
    Wj_sq <- diag(sqrt(w[idx[[g]]]))                                     # W_j^{1/2}
    Zj_w <- Wj_sq %*% Zj                                                 # W^{1/2} Z_j
    Vj[[g]] <- Zj_w %*% D %*% t(Zj_w) + as.numeric(s2) * diag(nrow(Zj_w))  # V_j
  }
  # Output:
  # list of length G with n_j x n_j covariance matrices V_j
  return(Vj)
}
```

Step 2

Given the new estimates of the fixed and random parts, the variance parameters are updated according to

$$\hat{\sigma}^{2(m)} = \frac{1}{N}\sum_{i=1}^{n}\left\{\hat{\varepsilon}_j^{(m)\top}\hat{\varepsilon}_j^{(m)} + \hat{\sigma}^{2(m-1)}\left[n_j - \hat{\sigma}^{2(m-1)}\,\mathrm{tr}\big(\hat{V}_j^{-(m-1)}\big)\right]\right\},$$

with residuals

$$\hat{\varepsilon}_j^{(m)} = W_j^{(M)1/2}\tilde{y}_j^{(M)} - W_j^{(M)1/2}\hat{f}^{(m)}(X_j) - W_j^{(M)1/2}Z_j\hat{b}_j^{(m)}.$$

In R:

```r
# sigma_fun: updates sigma^2 using paper's closed-form expression
sigma_fun <- function(N,    # N   : total number of observations
                      G,    # G   : number of clusters
                      idx,  # idx : list of row indices per cluster
                      b,    # b   : matrix q × n vector of random effects
                      y,    # y   : response vector length N
                      Z,    # Z   : N x q random-effects design
                      D,    # D   : q x q covariance of random effects
                      Vi,   # Vi  : list of length G with V_j matrices (n_j x n_j)
                      s2,   # s2  : scalar sigma^2 (residual variance; homoskedastic)
                      fhat, # fhat: current fixed-effect fit
                      w     # length N weights
                      ){
  term_sum <- 0
  for (g in seq_len(G)) {
    Zj <- Z[idx[[g]], , drop = FALSE]
    Wj_sq <- diag(sqrt(w[idx[[g]]]))                    # W_i^{1/2}
    e <- Wj_sq %*% y[idx[[g]]] - Wj_sq %*% fhat[idx[[g]]] - Wj_sq %*% Zj%*% b[g, ]       #  weighted re
    term_sum <- term_sum + crossprod(e) + s2 * (length(idx[[g]]) -
                        s2 * sum(diag(solve(Vi[[g]]))))     # add cluster-j contribution
  }
```

```r
  # Output:
  #   scalar sigma^2 new estimate
  term_sum / N                                          # divide by total N
}
```

$$\hat{D}^{(m)} = \frac{1}{n}\sum_{i=1}^{n}\left\{\hat{b}_j^{(m)}\hat{b}_j^{(m)\top} + \left[\hat{D}^{(m-1)} - \hat{D}^{(m-1)}(W_j^{(M)1/2}Z_j)^\top \hat{V}_j^{-(m-1)} W_j^{(M)1/2} Z_j \hat{D}^{(m-1)}\right]\right\},$$

In R:

```r
# D_fun: updates D using the paper's expression (average of b_j b_j^T plus shrinkage term)
D_fun <- function(G,   # G   : number of clusters
                  idx, # idx : list of row indices per cluster
                  b,   # b   : matrix q × n vector of random effects
                  Z,   # Z   : N x q random-effects design
                  D,   # D   : q x q covariance of random effects
                  Vj,  # Vj  : list of length G with V_j matrices (n_j x n_j)
                  w    # length N weights
                  ){
  q <- ncol(Z)
  S <- matrix(0, q, q)                                  # accumulator
  for (g in seq_len(G)) {
    Zj <- Z[idx[[g]], , drop = FALSE]                   # Z_j
    Wj_sq <- diag(sqrt(w[idx[[g]]]))                    # W_j^{1/2}
    Zj_w <- Wj_sq %*% Zj                                # W^{1/2} Z_j
    S <- S + tcrossprod(b[g, ]) +                       # b_j b_j^T
      (D - D %*% t(Zj_w) %*% solve(Vj[[g]]) %*% Zj_w %*% D)    # shrinkage term per cluster
  }
  # Output:
  #   q x q updated D
  S/G                                                   # average over clusters
}
```

Step 3

At each inner iteration, the following generalized log-likelihood (GLL) is computed:

$$\mathrm{GLL}\big(f(X_j), b_j \mid y\big) = \sum_{i=1}^{n}\left\{\hat{\varepsilon}_j^{(m)\top}(\hat{\sigma}^{2(m)}I_{n_j})^{-1}\hat{\varepsilon}_j^{(m)} + \hat{b}_j^{(m)\top}\hat{D}^{-(m)}\hat{b}_j^{(m)} + \log|\hat{D}^{(m)}| + \log|\hat{\sigma}^{2(m)}I_{n_j}|\right\}.$$

In R:

```r
# gll_fun: computes generalized log-likelihood (up to additive constant)
gll_fun <- function(idx, # idx : list of row indices per cluster
                    b,   # b   : matrix q × n vector of random effects
                    y,   # y   : response vector length N
                    Z,   # Z   : N x q random-effects design
                    D,   # D   : q x q covariance of random effects
                    s2,  # s2  : scalar sigma^2 (residual variance; homoskedastic)
                    fhat,# fhat: current fixed-effect fit
                    w    # length N weights
                    ){
```

```r
  logdetD <- as.numeric(determinant(D, logarithm = TRUE)$modulus)     # log|D|
  term_b <- sum(rowSums((b %*% solve(D)) * b))                         # sum_j b_j^T D^{-1} b_j
  term_r <- 0
  term_logRj <- 0
  for (i in seq_along(idx)) {
    Zj <- Z[idx[[i]], , drop = FALSE]                        # Z_i
    Wj_sq <- diag(sqrt(w[idx[[i]]]))                         # W_i^{1/2}
    e <- Wj_sq %*% y[idx[[i]]] - Wj_sq %*% fhat[idx[[i]]] - Wj_sq %*% Zj%*% b[i, ]       #  weighted re
    term_r <- term_r + sum(e^2) / s2                                   # (1/s2) * ||rj||^2
    term_logRj <- term_logRj + length(idx[[i]]) * log(s2)              # n_j * log(s2)
  }

  # Output:
  #   scalar GLL value (up to constant)
  # GLL = sum_j [ r_j^T R_j^{-1} r_j + b_j^T D^{-1} b_j + log|D| + log|R_j| ]
  term_r + term_logRj + term_b + length(idx) * logdetD                 # assemble GLL
}
```

End Inner loop

The inner loop continues until the relative change in the GLL between two successive iterations is smaller than a predefined tolerance threshold. To check convergence we use the Aitken stop, *this is not in the paper*, and I'm really unsure if it good enough, but looks like it (*Indeed for now it's not used anymore, still here because why not*)

In R:

```r
# ait_stop: Aitken acceleration based stopping rule
ait_stop <- function(gll,  #gll : numeric vector with GLL history (needs last three values)
                     tol,  #tol : tolerance for relative error to asymptote
                     it    #it  : current iteration index
                     ){
  converged <- FALSE
  l1 <- gll[it + 2] - gll[it + 1]                                      # L_t
  l2 <- gll[it + 1] - gll[it]                                          # L_{t-1}
  a <- l1 / l2                                                         # Aitken ratio
  if (is.finite(a) && a > 0 && a < 1) {                                # only meaningful if 0<a<1
    L_inf <- gll[it + 1] + l1 / (1 - a)                                # extrapolate asymptote
    err <- abs(L_inf - gll[it + 2]) / (abs(L_inf) + 1e-12)             # relative error to asymptote
    if (err < tol) converged <- TRUE                                   # declare convergence
  }
  converged                                                            # return flag
}
```

End Outer loop

After convergence of the inner loop at outer iteration M, the working quantities are updated to prepare the next outer iteration. The linear predictors are first recomputed as

$$\hat{\eta}_j^{(M+1)} = \hat{f}^{(m)}(X_j) + Z_j\,\hat{b}_j^{(m)}, \qquad j = 1, \ldots, n,$$

and the conditional means are obtained by applying the inverse of the logit link,

$$\hat{\mu}_j^{(M+1)} = g^{-1}\big(\hat{\eta}_j^{(M+1)}\big) = \frac{\exp\big(\hat{\eta}_j^{(M+1)}\big)}{1 + \exp\big(\hat{\eta}_j^{(M+1)}\big)}.$$

The new pseudo-responses and weights are then derived as

$$\tilde{y}_j^{(M+1)} = g\big(\hat{\mu}_j^{(M+1)}\big) + \big(y_j - \hat{\mu}_j^{(M+1)}\big)\, g'\big(\hat{\mu}_j^{(M+1)}\big), \qquad W_j^{(M+1)} = \mathrm{diag}\big(w_{ij}^{(M+1)}\big),$$

with $w_{ij}^{(M+1)} = \hat{\mu}_{ij}^{(M+1)}\big(1 - \hat{\mu}_{ij}^{(M+1)}\big)$. The index M is then increased by one, and the algorithm returns to the inner loop until the convergence of the linear predictors η_i across successive outer iterations.

Implementation

Here we put the algorithm in a single function with many parameter to be able to control the convergence and to be able to tune the tree from step 1.

```
fit_gmert    <- function(df,                # df: data.frame with columns
                                            #   id   : cluster identifier (factor or integer)
                                            #   y    : numeric response
                                            #   x1,x2,x3 : numeric covariates
                                            #   leaf : true generating leaf (optional, for diagnostics)
                          max_iter_inn = 1000,  # maximum number of EM iterations (inner loop)
                          max_iter_out = 1000,  # maximum number of PQL iterations (outer loop)
                          tol = 1e-6,           # convergence tolerance for both loops (Aitken or relati
                          cp = 0.0,             # rpart complexity parameter (pruning threshold)
                          minsplit = 50,        # minimum number of obs required to attempt a split
                          minbucket = 20,       # minimum number of obs in any terminal node
                          maxdepth = 5,         # maximum tree depth
                          xval = 10             # number of cross-validation folds in rpart
                          ) {

  # --- Basic setup ---
  N <- nrow(df)                                  # total number of observations
  G <- length(unique(df$id))                     # number of clusters
  idx_by_cluster <- split(seq_len(N), df$id)     # list: row indices grouped by cluster

  y <- df$y                                      # response vector
  Z <- cbind(1, df$x1)                           # random-effects design: intercept + slope on x1
  q <- ncol(Z)                                   # number of random effects (q = 2)

  # --- Initialization (Step 0) ---
  M <- 0L                                        # outer-loop counter
  mu <- ifelse(y == 1, 0.75, 0.25)               # initial conditional means
  y_t <- log(mu / (1 - mu)) + (y - mu) / (mu * (1 - mu))  # initial pseudo-response (PQL linearization)
  w <- mu * (1 - mu)                             # initial working weights
  sigma2 <- 1                                    # initial residual variance
  D <- diag(2)                                   # initial random-effects covariance (identity)
  b <- matrix(0, G, q)                           # initialize cluster random effects
  gll <- c(0, 0)                                 # GLL storage (2 slots for Aitken acceleration)
  eta_old <- rep(0, N)                           # previous eta for outer-loop convergence check
  converged_in <- c()                            # inner-loop convergence flags (per outer iteration)
  converged_out <- FALSE                         # outer-loop convergence flag

  # --- Outer loop (PQL updates) ---
  repeat {
    m = 0                                        # reset inner-loop counter
```

```r
# --- Inner loop (EM-like iterations) ---
repeat {
  m <- m + 1L

  # (1.i) Partial E-step: compute adjusted pseudo-response y_tilde* = y_tilde - Z b
  zb <- numeric(N)                          # cluster-specific random contributions
  for (g in seq_len(G)) {
    idx <- idx_by_cluster[[g]]              # indices for cluster g
    zb[idx] <- Z[idx, , drop = FALSE] %*% b[g, ]  # Z_i b_i
  }
  y_star <- y_t - zb                        # adjusted response for tree fit

  # (1.ii) M-step: fit regression tree for fixed effects f(X)
  ctrl <- rpart.control(cp = cp, minsplit = minsplit, xval = xval,
                        minbucket = minbucket, maxdepth = maxdepth)
  Xdf <- data.frame(x1 = df$x1, x2 = df$x2, x3 = df$x3)
  tree <- rpart(y_star ~ x1 + x2 + x3,
                data = cbind(y_star = y_star, Xdf),
                weights = w, method = "anova", control = ctrl)
  fhat <- as.numeric(predict(tree, newdata = Xdf))

  # (1.iii) Update random effects b_i
  Vi <- Vi_fun(Z = Z, D = D, s2 = sigma2, G = G, idx = idx_by_cluster, w = w)  # build V_i per clus
  b <- b_fun(G = G, Z = Z, Vi = Vi, D = D, idx = idx_by_cluster,
             y_t = y_t, fhat = fhat, w = w)                                  # update b_i estimate

  # (2.i) Update sigma^2 (residual variance)
  sigma2 <- sigma_fun(N = N, G = G, idx = idx_by_cluster,
                      b = b, y = y_t, Z = Z, D = D,
                      Vi = Vi, s2 = sigma2, fhat = fhat, w = w)

  # (2.ii) Update D (random-effects covariance)
  D <- D_fun(G = G, idx = idx_by_cluster,
             Z = Z, D = D, Vi = Vi, b = b, w = w)

  # --- Inner-loop convergence check (GLL stabilization) ---
  gll[m + 2] <- gll_fun(D = D, b = b, idx = idx_by_cluster,
                        Z = Z, y = y_t, fhat = fhat, s2 = sigma2, w = w)
  if (m > 1L) {
    rel <- abs(gll[m + 2] - gll[m + 1]) / (abs(gll[m + 1]) + 1e-12)
    if (rel < tol) { n_iter <- m; converged_in_t <- TRUE; break }
  }

  if (m >= max_iter_inn) {                  # max-iteration guard
    n_iter <- m
    converged_in_t <- FALSE
    # message(sprintf("WARNING: the EM algorithm did not converge in %d iterations.", max_iter_inn)
    break
  }
}

# --- Outer-loop update (PQL step) ---
converged_in <- c(converged_in, converged_in_t)
```

```r
    zb <- numeric(N)
    for (g in seq_len(G)) {
      idx <- idx_by_cluster[[g]]
      zb[idx] <- Z[idx, , drop = FALSE] %*% b[g, ]
    }
    eta <- fhat + zb                            # recompute linear predictor

    # (Outer stopping rule - paper style)
    d_eta <- sqrt(mean((eta - eta_old)^2))      # RMS change of eta
    if (d_eta < tol) {
      converged_out <- TRUE
      break
    }

    # Update working quantities for next outer iteration
    eta_old <- eta
    mu <- exp(eta) / (1 + exp(eta))             # updated conditional means
    y_t <- log(mu / (1 - mu)) + (y - mu) / (mu * (1 - mu))  # new pseudo-response
    w <- mu * (1 - mu)                          # new working weights

    M <- M + 1L
    if (M >= max_iter_out) {                    # guard against outer non-convergence
      converged_out <- FALSE
      message(sprintf("WARNING: the PQL algorithm did not converge in %d iterations.", max_iter_out))
      break
    }
  }

  # --- Return fitted components ---
  list(
    tree = tree,                    # fitted rpart tree (fixed-effects function f(X))
    b = b,                          # estimated random effects (G x q)
    D = D,                          # estimated random-effects covariance
    sigma2 = sigma2,                # estimated residual variance
    mu = mu,                        # conditional means
    converged_in = converged_in,    # convergence flags for inner loops
    converged_out = converged_out,  # convergence flag for outer loop
    n_iter = n_iter,                # iterations performed (inner loop)
    train_ids = df$id,              # cluster identifiers in training set
    gll = gll,                      # GLL trace (for diagnostics)
    tol = tol                       # convergence tolerance used
  )
}
```

Prediction

To predict the response for a new observation that belongs to a cluster among those used to fit the MERT model, we use both its corresponding population-averaged tree prediction and the predicted random part corresponding to its cluster. For a new observation that belongs to a cluster not included in the sample used to estimate the model parameters, we can only take the corresponding population-averaged tree prediction.

```r
predict_gmert <- function(fit, new_df, thr = 0.5
) {
  # fixed part: regression tree predictions using predictors only
  fhat <- as.numeric(predict(fit$tree, new_df))        # tree fitted on y*; use on new data
  # construct random effects design for new data: [1, x1]
  Znew <- cbind(1, new_df$x1)
  # container for random effects contributions
  add <- numeric(nrow(new_df))
  # clusters seen during training (so we have estimated b_i)
  clus_fit <- sort(unique(fit$train_ids))
  # map each new cluster id to its index in clus_fit (NA if unseen)
  map <- match(new_df$id, clus_fit)
  seen <- !is.na(map)                                  # TRUE for rows belonging to seen clusters
  if (any(seen)) {
    # add random effect contribution: row-wise sum of [1, x1] * b_i
    add[seen] <- rowSums(Znew[seen, , drop = FALSE] *
                         fit$b[map[seen], ])
  }
  # total prediction = fixed part + random effects contribution
  eta   <- fhat + add
  p     <- 1 / (1 + exp(-eta))

  # return 0/1;
  yhat  <- ifelse(p >= thr, 1, 0)
  yhat
}
```